



Lets First understand AWS Lambda with real world example:

Imagine a Restaurant;

- **Customer (Event Source)** → Orders food.
- **Chef (Lambda Function)** → Prepares the dish only when an order comes.
- **Kitchen (AWS Infrastructure)** → Provides stove, utensils (runtime environment).
- **Waiter (Trigger)** → Brings the order request to the chef.

👉 Key point: The chef (Lambda) doesn't sit idle all day. He works only when an order comes (event-driven).

Now,

#AWS Lambda is a compute service that lets you run your code in response to events without the need to provision or manage servers.

-It scales Automatically.

-Serverless .

-you only pay for execution time.

AWS LAMBDA FEATURES

1. **Event-Driven** → Lambda runs only when something happens (like file upload, API request, or a timer).
2. **No Servers to Manage** → You just write code, AWS runs it for you.
3. **Auto Scaling** → Handles 1 request or 1 million requests without extra setup.
4. **Pay Only When Used** → You're charged only while your code runs.
5. **Multi-Language Support** → Works with Python, Node.js, Java, Go, and more.
6. **AWS Lambda Integration Support** seamlessly integrates with other AWS services like API Gateway, DynamoDB, S3, Step Functions, SNS, and SQS to build fully event-driven applications.

AWS Lambda Concepts

Function → Your actual code (like a small program) that runs when triggered.

Runtime → The programming language environment (Python, Node.js, Java, etc.) in which your function runs.

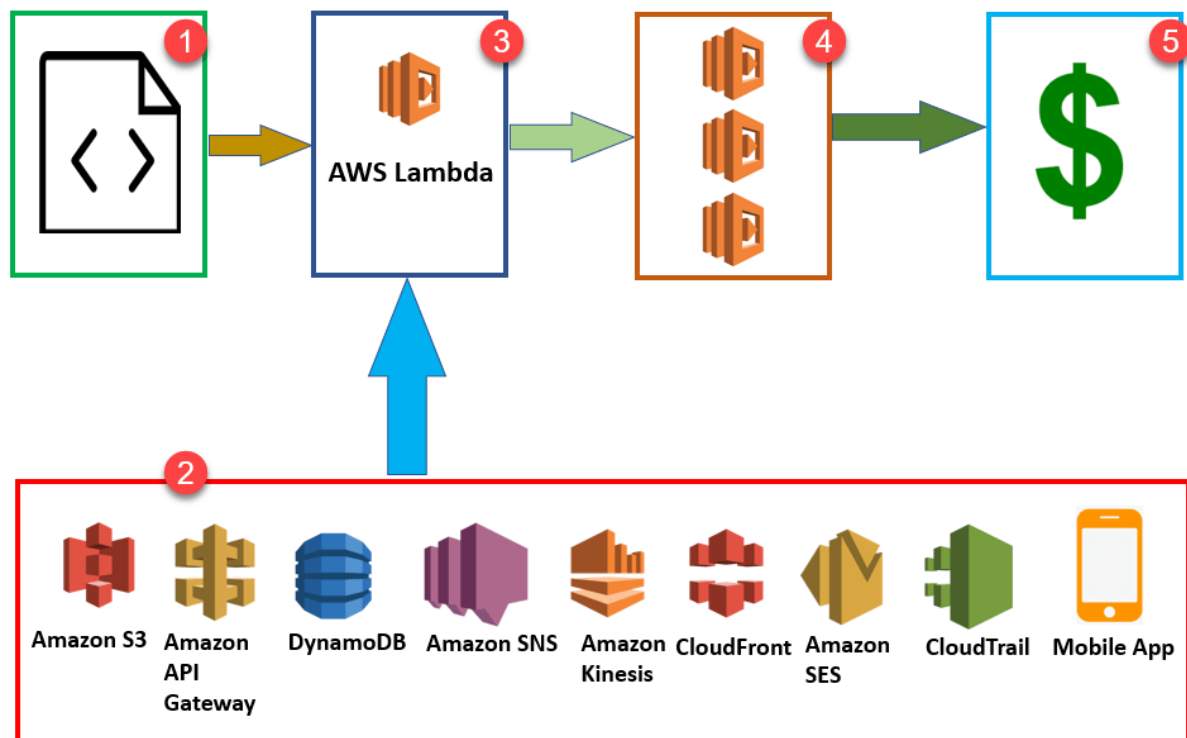
Event Source → The thing that triggers your function (like S3 upload, API request, or SNS message).

Layers → A way to add extra libraries or code (like plugins) that your function needs without writing them inside the main function.

Log Streams → Logs (via CloudWatch) that record what your function did, errors, and performance details.

How does AWS Lambda work?

The following AWS Lambda example with block diagram explains the working of AWS Lambda in a few easy steps:



AWS Lambda Block Diagram

Step 1: First upload your AWS Lambda code in any language supported by AWS Lambda. [Java](#), Python, Go, and C# are some of the languages that are supported by AWS Lambda function.

Step 2: These are some AWS services which allow you to trigger AWS Lambda.

Step 3: AWS Lambda helps you to upload code and the event details on which it should be triggered.

Step 4: Executes AWS Lambda Code when it is triggered by AWS services:

Step 5: AWS charges only when the AWS lambda code executes, and not otherwise.

This will happen in the following scenarios:

- Upload files in an S3 bucket
- When HTTP get/post endpoint URL is hit
- For adding/modifying and deleting Dynamo DB tables
- In the process of data streams collection
- Push notification
- Hosting of website
- Email sending

Note: You should remember that you will charge for AWS services only when the AWS Lambda code executes, else you don't need to pay anything.

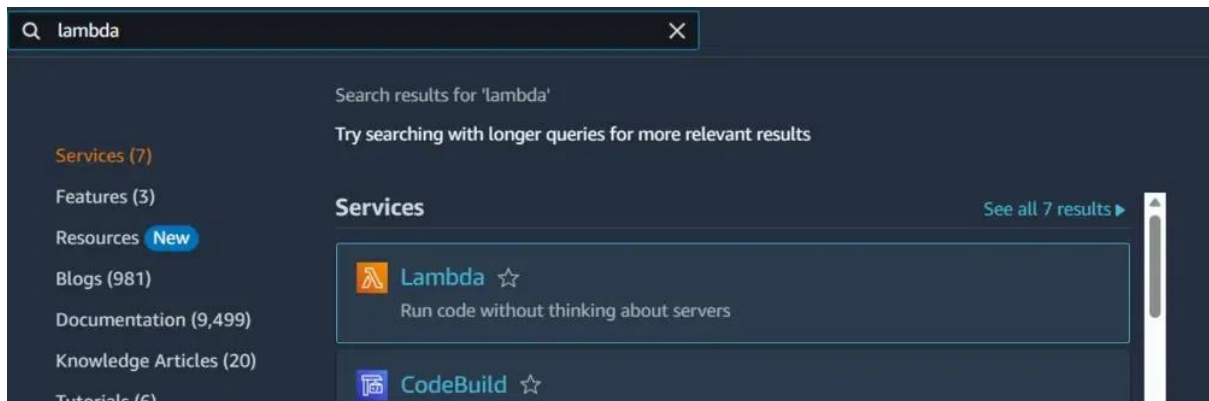
Getting Started With AWS Lambda Function

A Lambda function is a small block of code that runs in response to events. functions are the basic building blocks you use to build applications. Function handlers are the entry point for event objects that your Lambda function code processes.

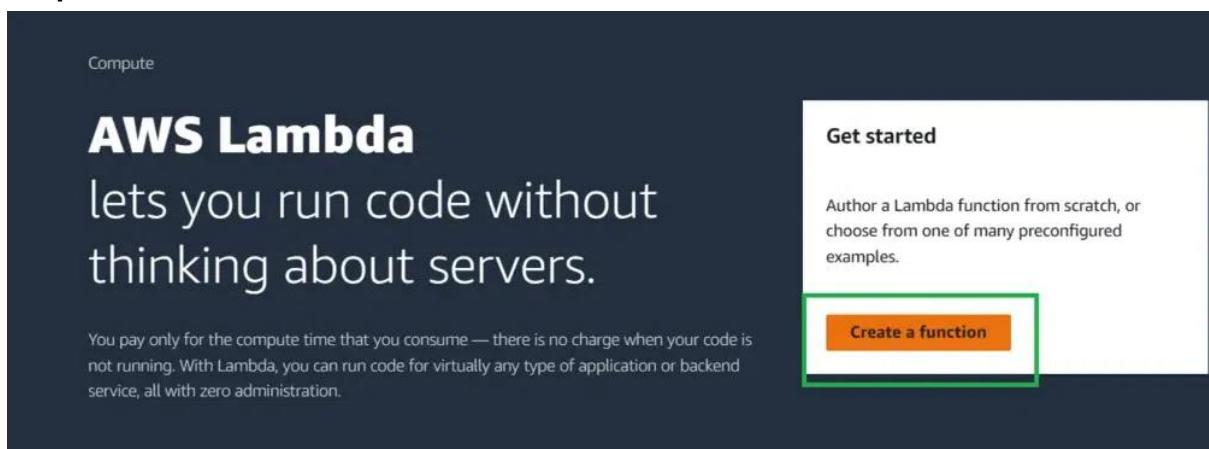
#The following are the steps mentioned below to create your own customized AWS lambda functions by using the AWS console. [Create an AWS account.](#)

Steps for Creating AWS Lambda Functions Using AWS Console:

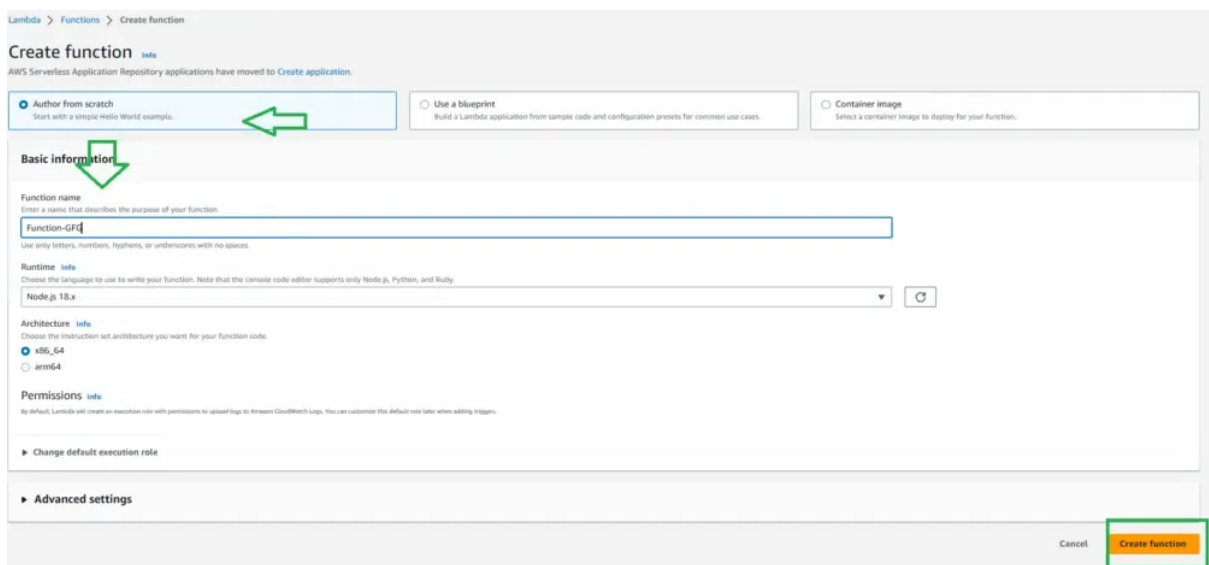
Step 1: Log in to your AWS console and search for Lambda. As shown in the following image.



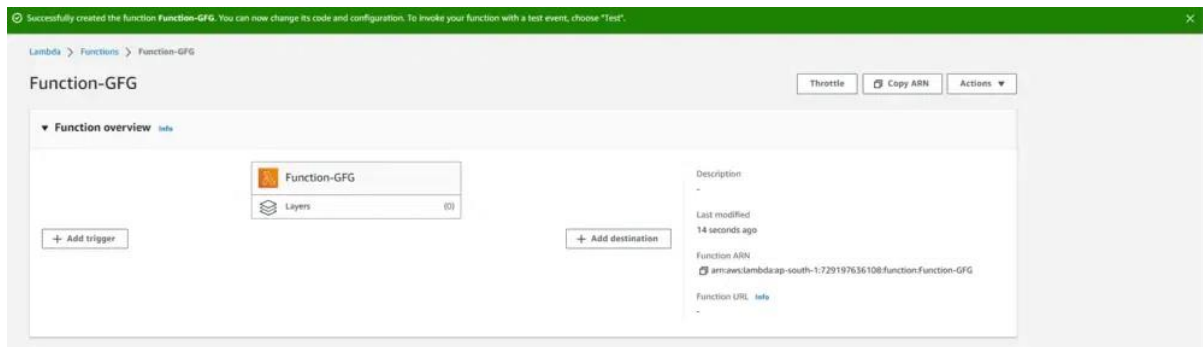
Step 2: Click on Create function.



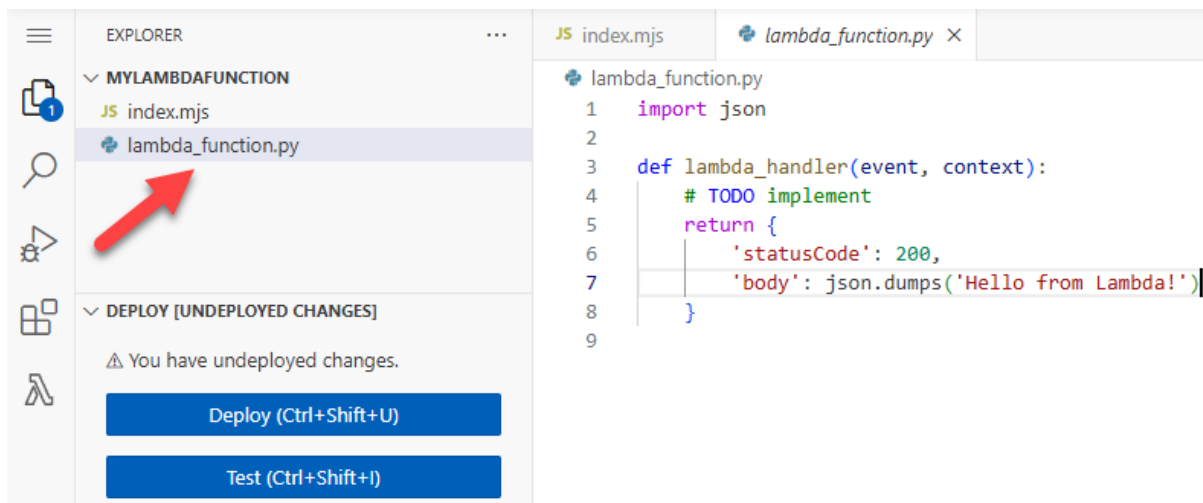
Step 3: Here we are going to use a sample hello world program by using Author from scratch and configure the details according to your requirement.



Step 4: Successfully our function is created.



Step 5: In the console's built-in code editor, you should see the function code that Lambda created. If you don't see the **lambda_function.py** tab in the code editor, select **lambda_function.py** in the file explorer as shown on the following diagram.



Paste the following code into the `lambda_function.py` tab, replacing the code that Lambda created.

```
import json
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):

    # Get the length and width parameters from the event object. The
    # runtime converts the event object to a Python dictionary
    length = event['length']
    width = event['width']

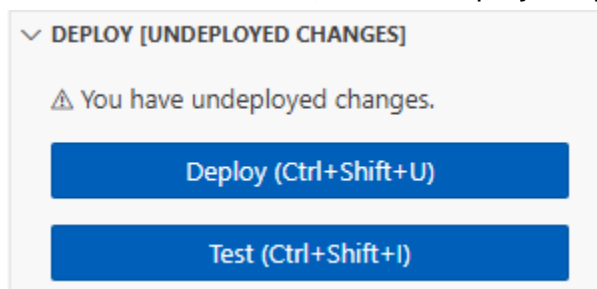
    area = calculate_area(length, width)
    print(f"The area is {area}")

    logger.info(f"CloudWatch logs group: {context.log_group_name}")

    # return the calculated area as a JSON string
    data = {"area": area}
    return json.dumps(data)

def calculate_area(length, width):
    return length*width
```

In the DEPLOY section, choose Deploy to update your function's code:



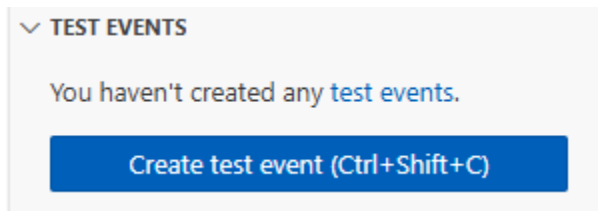
Next,

Invoke the Lambda function using the console code editor

To invoke your function using the Lambda console code editor, create a test event to send to your function. The event is a JSON formatted document containing two key-value pairs with the keys "length" and "width".

To create the test event;

1. In the TEST EVENTS section of the console code editor, choose Create test event.

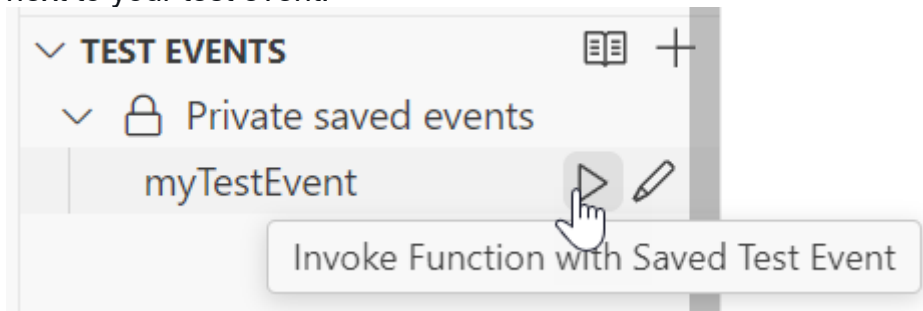


2. For **Event Name**, enter `myTestEvent`.
3. In the **Event JSON** section, replace the default JSON with the following:

```
{  
  "length": 6,  
  "width": 7  
}
```
4. Choose **Save**.

To test your function and view invocation records

In the **TEST EVENTS** section of the console code editor, choose the run icon next to your test event:



When your function finishes running, the response and function logs are displayed in the **OUTPUT** tab. You should see results similar to the following:

Node.js

Python

Status: Succeeded 

Test Event Name: myTestEvent

Response
{"area": 42}

Function Logs
START RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b Version: \$LATEST
The area is 42
[INFO] 2024-08-31T23:43:26.428Z 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b CloudW
END RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b
REPORT RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b Duration: 1.42 ms Billed

Request ID
2d0b1579-46fb-4bf7-a6e1-8e08840eae5b

When you invoke your function outside of the Lambda console, you must use CloudWatch Logs to view your function's execution results.

To view your function's invocation records in CloudWatch Logs

1. Open the [Log groups](#) page of the CloudWatch console.
2. Choose the log group for your function (/aws/lambda/myLambdaFunction). This is the log group name that your function printed to the console.
3. Scroll down and choose the Log stream for the function invocations you want to look at.

Log streams (14)			Delete	Create log stream	Search all log streams
Filter log streams or try prefix search		☐ Exact match ☐ Show expired Info < 1 > 			
☐	Log stream	Last event time			
☐	2024/04/30/[\$LATEST]e0fa	2024-04-30 17:24:16 (UTC)			
☐	2024/04/19/[\$LATEST]e9a	2024-04-19 20:59:06 (UTC)			
☐	2024/02/22/[\$LATEST]cf0	2024-02-22 18:38:41 (UTC)			
☐	2024/02/21/[1]d132c4d	2024-02-21 21:37:01 (UTC)			
☐	2024/02/21/[1]5ad	2024-02-21 21:37:01 (UTC)			

You should see output similar to the following:

A screenshot of the AWS Lambda console interface. At the top, there are two tabs: 'Node.js' and 'Python', with 'Python' being the active tab. Below the tabs is a log viewer showing the execution logs of a Lambda function. The logs include the following text: 'INIT_START Runtime Version: python:3.13.v16 Runtime Version ARN: arn:aws:lambda:', 'START RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e Version: \$LATEST', 'The area is 42', '[INFO] 2024-09-01T00:05:22.464Z 9315ab6b-354a-486e-884a-2fb2972b7d84 Clc', 'END RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e', and 'REPORT RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e Duration: 1.15 ms Bill'. The log viewer has a horizontal scrollbar at the bottom.

Pricing of AWS Lambda Function:

AWS lambda pricing is based on the no.of requests made to your function and the time it has taken to execute the function.

Requests → Charged per number of function invocations (1M requests free/month).

Duration → Charged based on execution time + memory allocated (400,000 GB-seconds free/month).

Free Tier → Always includes 1M free requests + 400K GB-seconds.

No cost to create functions, you only pay when they run.

Difference between Ec2 and Lambda

1)EC2 gives you full virtual servers where you manage OS, scaling, and runtime, while **Lambda** is serverless and runs code only when triggered without managing infrastructure.

2)EC2 runs continuously, whereas **Lambda** runs on-demand and charges only for execution time.

